

COMP 3311 DATABASE MANAGEMENT SYSTEMS

LECTURE 1 DATABASE MANAGEMENT SYSTEMS

DATABASE MANAGEMENT SYSTEMS: OUTLINE

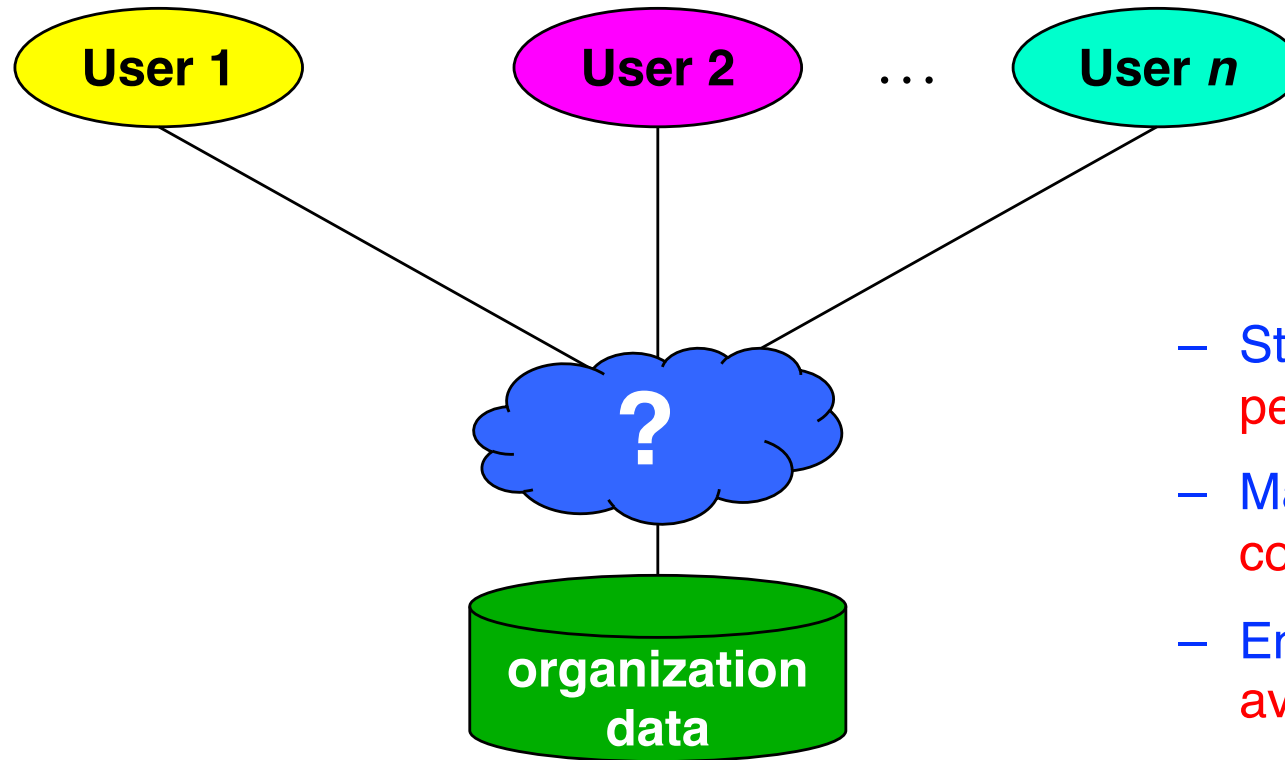
What Is A Database Management System (DBMS)?

Why Do We Need Database Management Systems?

How Does A DBMS Manage Data?

What Are The Major Components of A DBMS?

THE GENERAL PROBLEM WE WANT TO ADDRESS



Goals

- Store data persistently
- Maintain data consistency
- Ensure data availability

How best to manage an organization's data
that is stored in a computer?

👉 organize, access, share, protect,

DATABASE

A database is a collection of, usually, related data.

Data are facts such as age, salary, name, social media posts, etc.

- A database has the following properties.
 - It is designed, built and populated with data for a specific purpose.
Applications: sales, human resources, manufacturing, banking, real estate, stock trading, inventory management, social media, ride sharing, ...
 - It usually represents some aspect of the real world about which we want to store and manage data.
 - The data have some inherent meaning (semantics).

 **Databases touch all aspects of our lives!**

DATABASE MANAGEMENT SYSTEM (DBMS)

A database management system (DBMS) is a general-purpose software system used to manage databases.

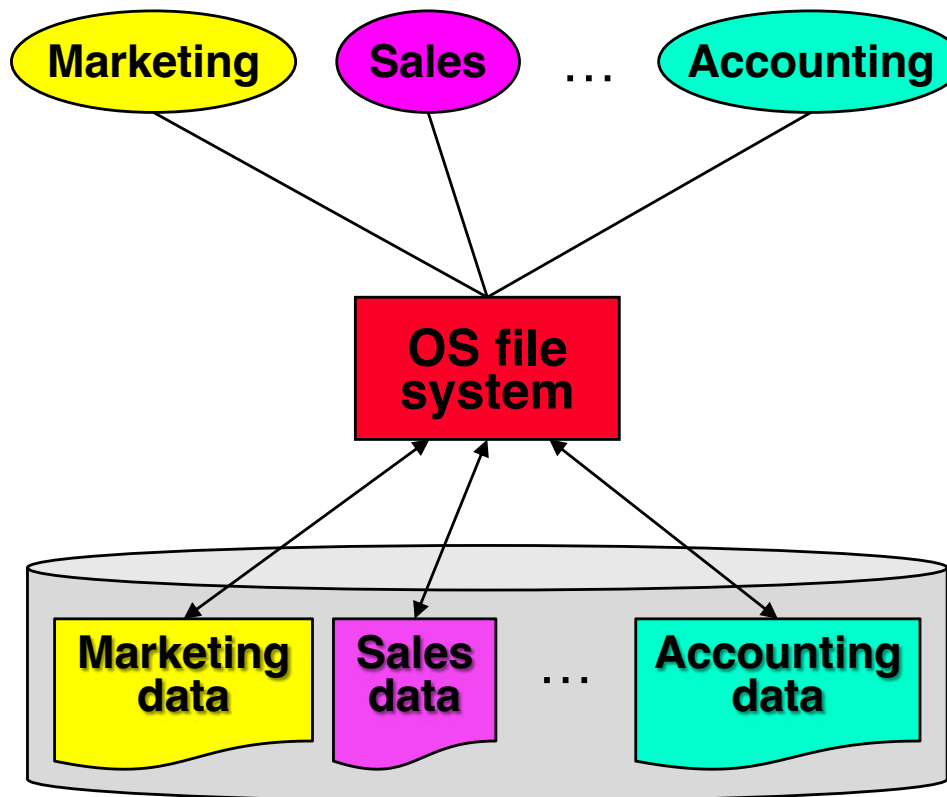
- A DBMS usually provides support/facilities for:
 - **defining** what data to store (with possibly types, relationships, constraints)
 - **storing** and **managing** data on a computer storage device (usually disk)
 - **manipulating** data (accessing, updating)
 - **sharing** data among many users
 - **protecting** data from loss, corruption, unauthorized access, ...

<i>Company</i>	<i>Product</i>	Relational
IBM	DB2	
Microsoft	SQL Server	
Oracle	Oracle Database	Non-relational
Amazon	DynamoDB	
Apache	Kudu	
MongoDB	MongoDB	
Neo4J	Neo4J	

A DBMS provides a software environment for managing data that is both *convenient* and *efficient* to use.

DATA MANAGEMENT USING AN OS FILE SYSTEM

Applications access stored data using the facilities provided by an operating system (OS) file system.



Drawbacks

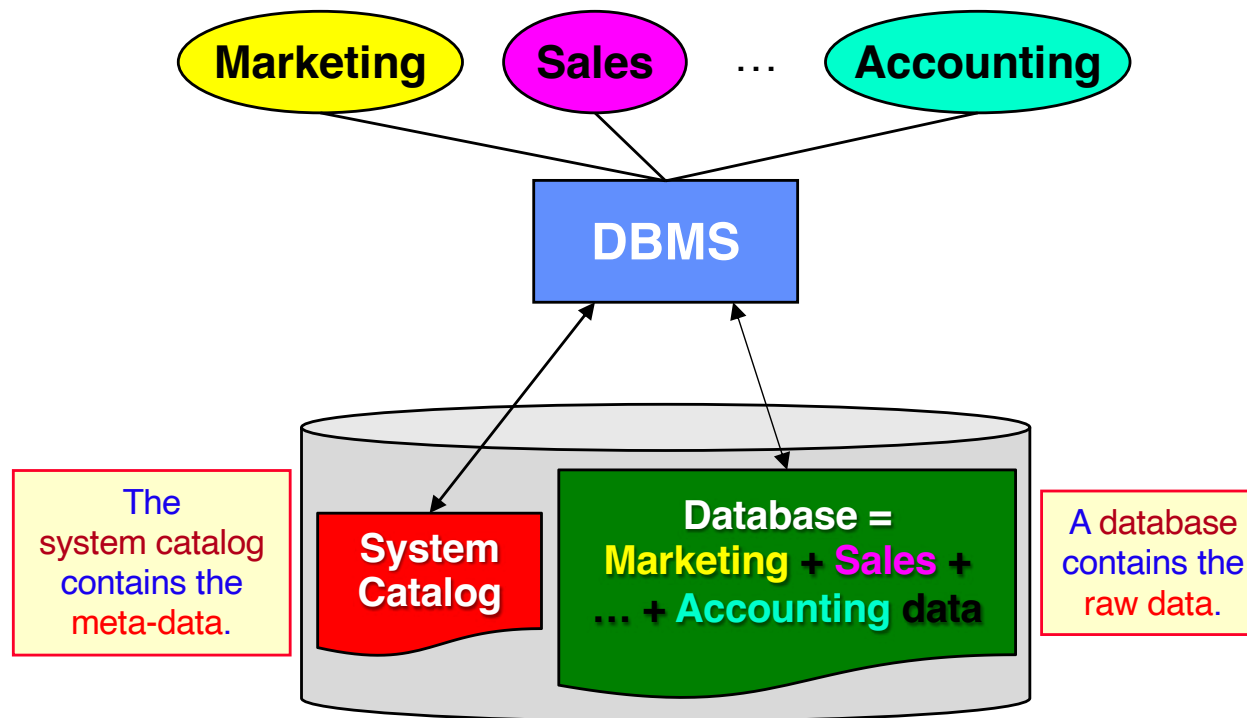
- Data duplication and inconsistency
- Difficulty meeting unanticipated needs
- Isolation of related data
- Constraint issues
- Atomicity of updates
- Concurrent access issues
- Data security problems

DATA MANAGEMENT USING A FILE SYSTEM: DRAWBACKS

- **Data duplication and inconsistency**
 - Data may be duplicated in several files possibly leading to data inconsistency.
- **Difficulty meeting unanticipated needs**
 - Data are structured to meet the needs of specific applications; unanticipated data needs often require writing new application programs to extract the required data.
- **Isolation of related data**
 - Related data are often scattered in different files and in different formats making writing new application programs requiring related data challenging.
- **Constraint issues**
 - Constraints on data become part of the program code making them hard to change and making it hard to know what constraints apply.
 - New constraints are often hard to enforce especially if they apply across several files.
- **Atomicity of updates**
 - Since changes to data may involve several updates (e.g., a transfer of funds between bank accounts), hardware/software failures can cause data to become inconsistent.
- **Concurrent access issues**
 - Data may become lost or inconsistent due to simultaneous access by many users (e.g., lost update).
- **Data security problems**
 - Enforcing security of data can be challenging due to *ad hoc* data management.

DATA MANAGEMENT USING A DBMS

Applications **access stored data** using only the facilities provided by a **DBMS**.



Major Principles

- **integrates** and **unifies** an organization's **data**.
- allows **separation** of **meta-data** (description of data) and **raw data** (facts).
- **supports** multiple users of the data.
- **controls** definition and access of data **centrally**.

A DBMS provides automated solutions for the data management problems encountered when using OS file systems.

DATA MODELS

A **data model** is the language used by a DBMS to describe how data is organized and accessed.

It defines how users **view** and **interact** with the data in a database.

A data model usually consist of:

1. **data structure types** $\Rightarrow$ specify **logical data organization** (**properties**, **relationships** and **semantics**)
2. **integrity constraints** $\Rightarrow$ specify **restrictions** on **properties** of the data and the **relationships** among the data
3. **operations** $\Rightarrow$ specify how data is **accessed** (e.g., R,I,U,D—Read, Insert, Update, Delete)

A data model is used to describe and organize data as well as to possibly state restrictions on the data.

SCHEMAS AND INSTANCES

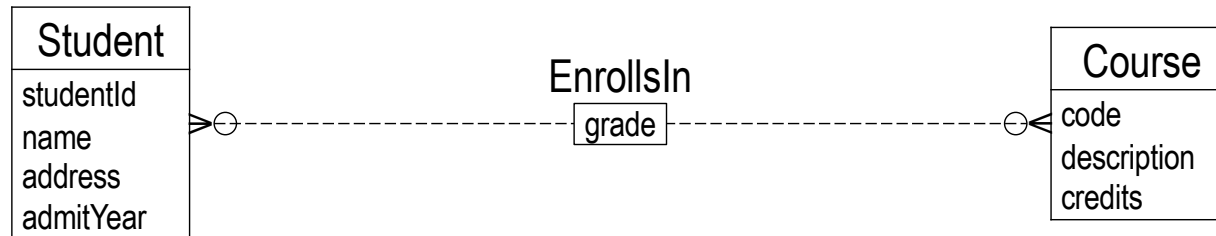
- A **database schema** is a **description of (some/all) the data in a database according to a data model.**
 - ✎ **Stored in the system catalog;
changes infrequently, if at all.**
- A **database instance** refers to the **actual content** of the database at a particular **point in time.**
 - ✎ **Conforms to the corresponding database schema, if present;
changes frequently as the data change.**

DATA MODEL, SCHEMA, INSTANCES EXAMPLES

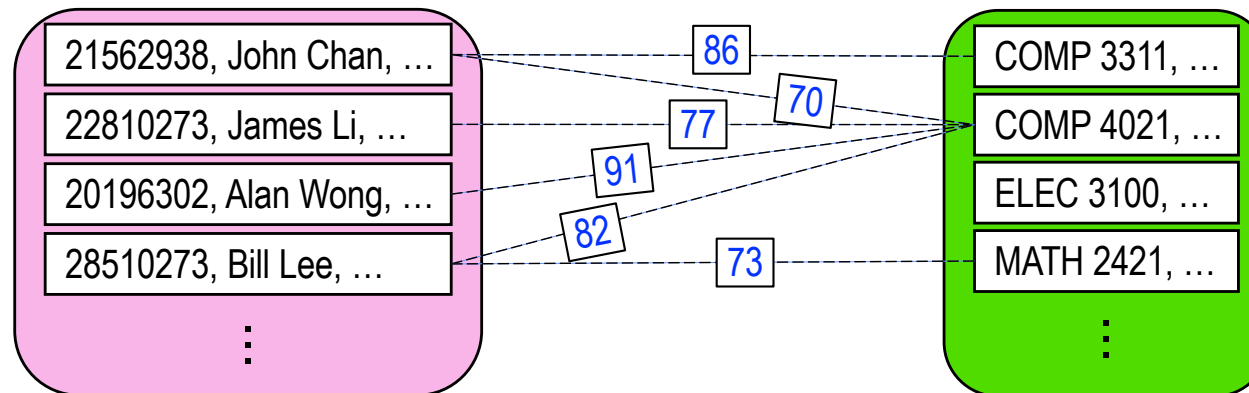
Entity-Relationship (E-R)

The data semantics are in the schema.

schema



database instance



👉 **Users view data as entities and relationships among the entities; relationships are represented explicitly in the schema; a database instance conforms to the schema.**

DATA MODEL, SCHEMA, INSTANCES EXAMPLES

(CONTD)

The data semantics are in the schema.

Relational

schema

Student

studentId	name	address	admitYear
-----------	------	---------	-----------

EnrollsIn

studentId	code	grade
-----------	------	-------

Course

code	description	credits
------	-------------	---------

database
instance

21562938	John Chan	...
22810273	James Li	...
20196302	Alan Wong	...
28510273	Bill Lee	...
⋮		

21562938	COMP 3311	86
20196302	COMP 4021	91
21562938	COMP 4021	70
28510273	MATH 2421	73
22810273	COMP 4021	77
28510273	COMP 4021	82
⋮		

COMP 3311	...
COMP 4021	...
ELEC 3100	...
MATH 2421	...
⋮	

👉 Users view data as tables;
relationships are represented implicitly in the data;
a database instance conforms to the schema.

DATA MODEL, SCHEMA, INSTANCES EXAMPLES

(CONTD)

The data semantics are embedded in the applications.

Key-Value

database
instance

key	value
'21562938'	'John Chan'
	⋮
	'COMP 3311', 86
	'COMP 4021', 91
'22810273'	'James Li'
	⋮
	'COMP 4021', 77
⋮	

key	value
'COMP 3311'	...
'COMP 4021'	...
'ELEC 3100'	...
'MATH 2421'	...
⋮	

👉 Users view data as key-value pairs;
relationships are aggregated in the data;
there is no schema.

DATA MODEL, SCHEMA, INSTANCES EXAMPLES

(CONT'D)

The data semantics are embedded in the applications.

Key-Value

database
instance

key	value
'21562938'	'John Chan'
	⋮
'22810273'	'James Li'
	⋮
	⋮

key	value
'COMP 3311'	⋮
	'21562938', 86
'COMP 4021'	⋮
	'21562938', 70
	'22810273', 77
	'20196302', 91
	'28510273', 82
	⋮

👉 Users view data as key-value pairs;
relationships are aggregated in the data;
there is no schema.

DATA MODEL, SCHEMA, INSTANCES EXAMPLES

(CONTD)

Document

database
instance

The data semantics are
embedded with the data.

<pre>{_id: '21562938', name: 'John Chan', ..., enrolledIn: [{code: 'COMP 3311', grade: 86}, {code: 'COMP 4021', grade: 91}] }</pre>
<pre>{_id: '22810273', name: 'James Li', ..., enrolledIn: [{code: 'COMP 4021', grade: 77}] }</pre>
<pre>...</pre>

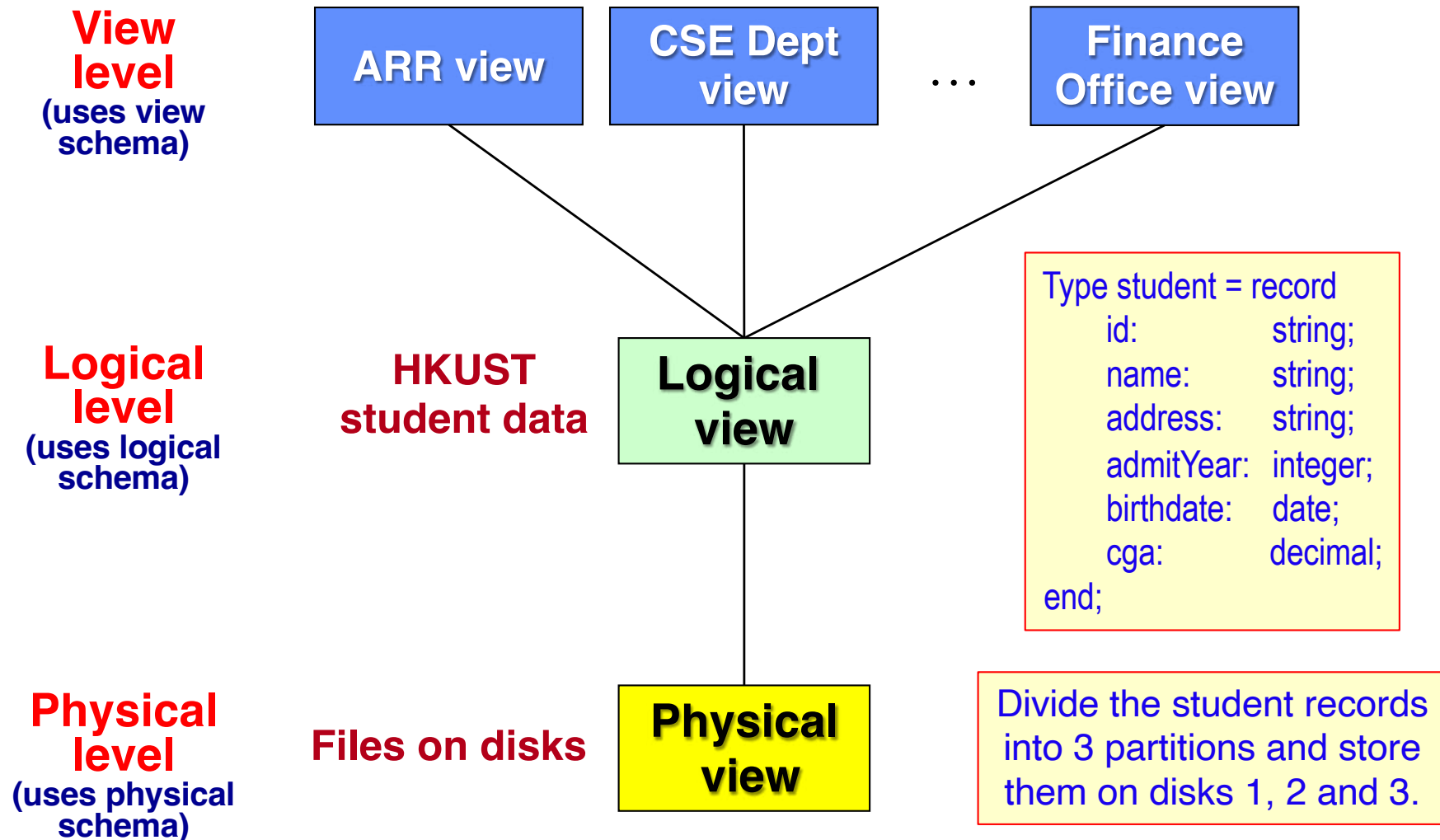
👉 **Users view data as documents;**
relationships are aggregated in the data;
there is no schema.

APPLICATION/DATA DEPENDENCE

- A big problem in application development is often the *dependence* of applications (i.e., the code) on the data that they access.
- **Do I have to change my application when I ...**
 - replace my disk (i.e., move the data to a new disk)?
 - partition the data into two physical files (or merge two physical files into one)?
 - store salary as a floating-point number instead of an integer?
 - develop other applications that use the same data?
 - add more data fields to support other applications?
 - index the data using a B⁺-tree instead of a hash index?

A DBMS can provide some degree of application/data independence using levels of data abstraction.

PROVIDING DATA INDEPENDENCE: THREE LEVELS OF DATA ABSTRACTION



PROVIDING DATA INDEPENDENCE:

THREE LEVELS OF DATA ABSTRACTION (CONT'D)

View level: Defines a subset (part) of the database for a particular application using a view schema.

- Views can hide information for security/privacy purposes (e.g., cga) or add information not actually stored (e.g., age).

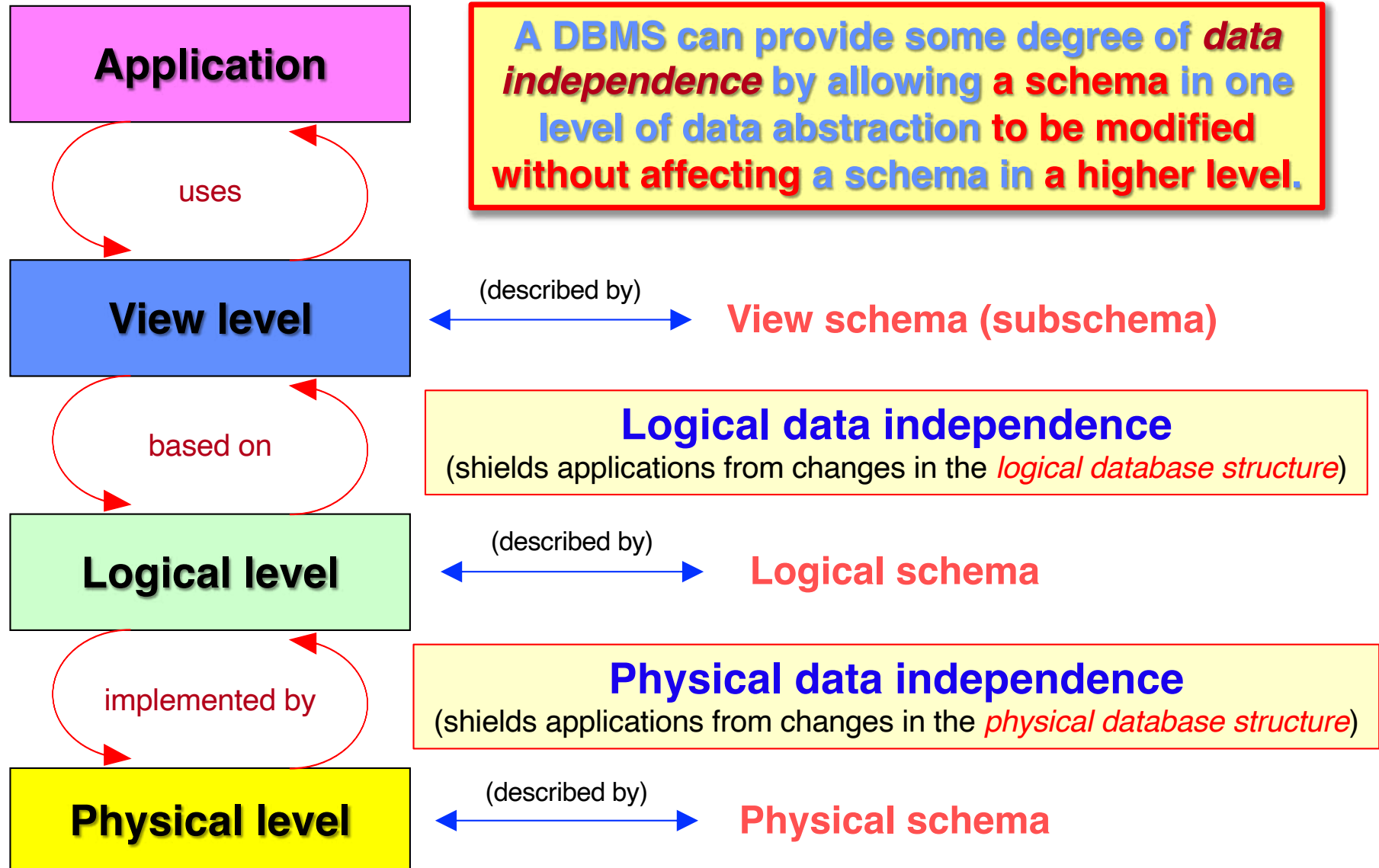
Logical level: Describes what data are stored in the database and the relationships among the data using a logical schema.

- E.g., students are related to courses and to fee information.
- Hides the details of how the data are physically stored from users.

Physical level: Describes how the data are stored on disk using a physical schema.

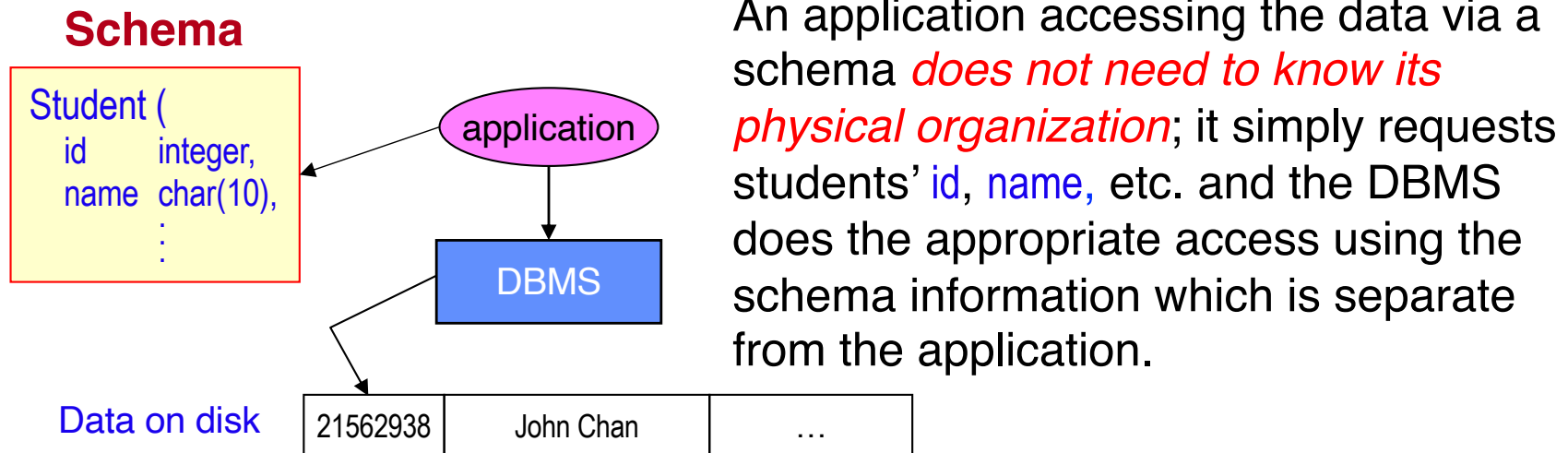
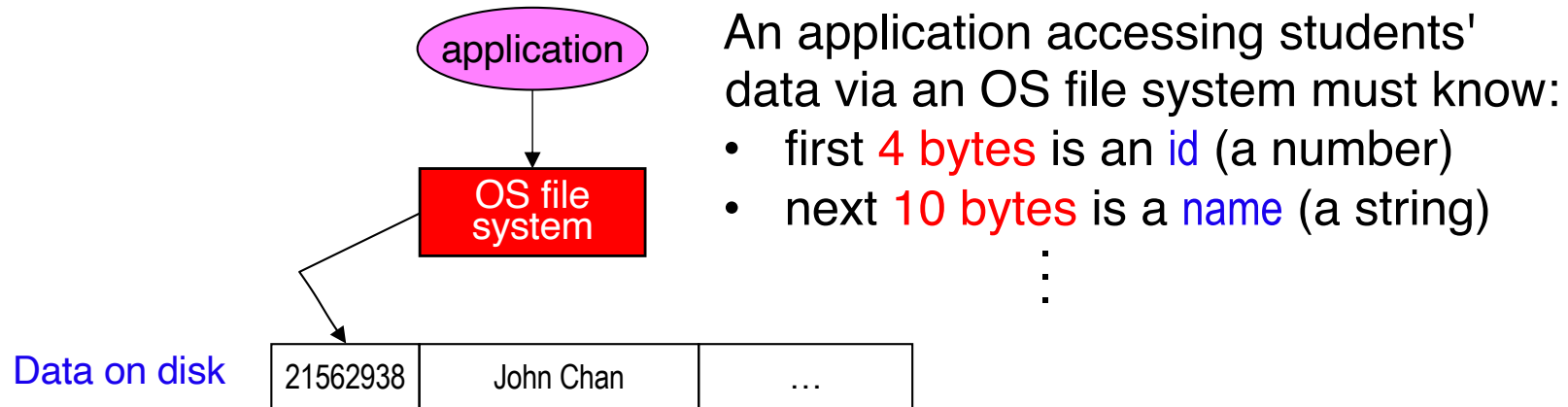
- Divide the student records into 3 partitions and store them on disks 1, 2 and 3.

PROVIDING DATA INDEPENDENCE VIA SCHEMAS



PROVIDING DATA INDEPENDENCE VIA SCHEMAS:

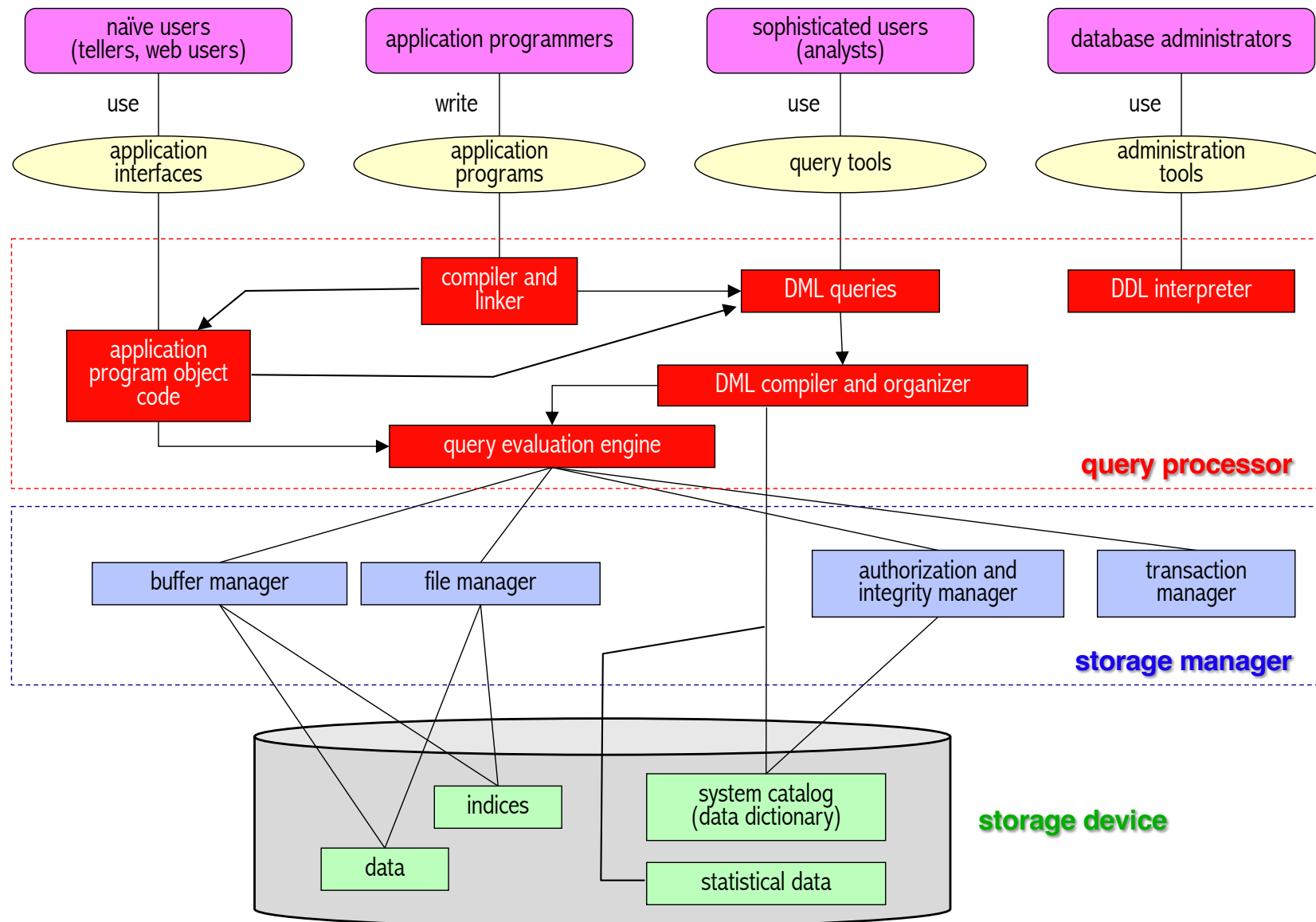
EXAMPLE



DATA INDEPENDENCE AND DATABASE SYSTEMS

- **Relational DBMSs** provide data independence via schemas following the three levels of data abstraction approach.
 - The **data semantics** are **represented in the schema** (the description) and are **stored separately from the data** (the facts) in the **system catalog**.
 - All data in the database **must conform to the schema**.
 - Application programs and queries are usually **simpler to develop**.
- **Non-relational (NoSQL) DBMSs** generally do not use schemas (i.e., they are **schemaless**).
 - The **data semantics** are either **encoded in the applications** (i.e., the code) that access the database or are **embedded within the data**.
 - The lack of a schema allows for **flexibility in what data is stored**.
 - Application programs and queries are usually **more complex to develop**.

GENERAL DBMS ARCHITECTURE



GENERAL DBMS ARCHITECTURE:

STORAGE MANAGER

- The **storage manager** provides the **interface** between the **raw data** stored in the database (i.e., on physical storage devices) and the **application programs/queries** that access the database.

 **It is responsible for storing, retrieving and updating data in the database.**

- The storage manager components usually include:
 - **buffer manager** (*fetches data from secondary storage and caches it in main memory*)
 - **file manager** (*manages allocation of secondary storage*)
 - **authorization and integrity manager** (*controls database access; enforces constraints*)
 - **transaction manager** (*ensures database consistency despite system failures and concurrent access by multiple users*)

GENERAL DBMS ARCHITECTURE:

QUERY PROCESSOR

- The **query processor** translates application requests (i.e., queries and updates) into efficient device-level operations.
 - ✎ It facilitates data access while providing good performance.
- The query processor component includes either one or two query languages:
 - **data definition language (DDL)** defines the database schema according to a data model. (Not relevant for most NoSQL DBMSs.)
 - **data manipulation language (DML)** accesses and manipulates data.
 - **Procedural DML**: the user specifies both *what* data is required and *how* to get the data.
 - Many non-relational DBMSs provide only a basic DML (e.g., get/put data).
 - **Nonprocedural (declarative) DML**: the user specifies only *what* data is required not how to get the data
 - All relational DBMSs provide **Structured Query Language (SQL)**, a nonprocedural DML.
 - Some non-relational DBMSs provide a DML similar to SQL.

GENERAL DBMS ARCHITECTURE:

QUERY PROCESSOR

Non-procedural DML: Structured Query Language (SQL)

schema

Student

studentId	name	address	admitYear
-----------	------	---------	-----------

EnrollIn

studentId	code	grade
-----------	------	-------

Course

code	description	credits
------	-------------	---------

Query: Find the minimum, maximum and average grade for COMP 3311.

```
select min(grade), max(grade), avg(grade)
from EnrollIn
where code='COMP 3311';
```

GENERAL DBMS ARCHITECTURE:

QUERY PROCESSOR

Procedural DML: MongoDB Query Language (MQL)

Query: Find the minimum, maximum and average grade for COMP 3311.

```
db.students.aggregate([
  {$unwind: {path: '$enrolledIn'}},
  {$match: {'enrolledIn.code': {$eq: 'COMP 3311'}}},
  {$group: {
    _id: null,
    minimum: {$min: '$enrolledIn.grade'},
    maximum: {$max: '$enrolledIn.grade'},
    average: {$avg: '$enrolledIn.grade'}
  }},
  {$project: {_id: 0}}
]);
```

{_id: '21562938', name: 'John Chan', enrolledIn: [{code: 'COMP 3311', grade: 86}, {code: 'COMP 4021', grade: 91}]}
{_id: '22810273', name: 'James Li', enrolledIn: [{code: 'COMP 4021', grade: 77}]}

GENERAL DBMS ARCHITECTURE: USERS

End Users

Naïve users

- Use existing application programs (e.g., print monthly sales report).
- Interact with applications through a graphical user interface (GUI).

Application programmers

- Develop applications that access a DBMS using the DML.

Sophisticated users

- Issue queries either directly using a DBMS query language or via tools such as data analysis software.

Database Administrator (DBA)

- Coordinates all activities of the DBMS.
 - Defines and maintains the schemas, if any.
 - Defines and maintains the physical organization.
 - Monitors and optimizes the database performance.
 - Monitors access and grants access rights.

A DBA must have a good understanding of an enterprise's information resources and needs.

DATABASE MANAGEMENT SYSTEMS: SUMMARY

- **Database management systems (DBMSs)** address the limitations of OS file systems for managing an organization's data.
- **Data models** are the foundation for developing a database—the entity-relationship (E-R) model and relational model are commonly used in practice for structured data while key-value and document models are used for less structured data.
- **Data independence** is fundamental to understanding how relational DBMSs manages data at different data abstraction levels. It is not a major concern for applications that use non-relational (NoSQL) DBMSs.
- **A DBMS provides many facilities** for data management and data access to efficiently handle the data needs of a variety of users and applications.